

DSA STUDY BLUEPRINT SERIES

43. Palindrome Partitioning Backtracking

Partitioning String into Palindromic Substrings

Section 06 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Problem:** Partition string such that every substring is a palindrome.
- Partition string recursively and validate boundaries.

Memory & Execution Blueprint

"aab" $\Rightarrow$ ["a", "a", "b"] or ["aa", "b"]

C++ Implementation Reference

Standard code layout and syntax

```
void partition(string s, int start, vector& curr, vector<vector>& ans) {  
    if (start == s.length()) { ans.push_back(curr); return; }  
    for (int end = start; end < s.length(); end++) {  
        if (isPalindrome(s, start, end)) {  
            curr.push_back(s.substr(start, end - start + 1));  
            partition(s, end + 1, curr, ans);  
            curr.pop_back(); // Backtrack  
        }  
    }  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N * 2^N)$
Space Complexity	$O(N)$

Key Practices & Gotchas

- **Important:** Use memoization/DP to check palindrome substrings in $O(1)$ time.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace recursive decision tree transitions generating subsets for [1, 2] :

Decision Level	Element choice	Recursive action	Subsets array state
i = 0	Include 1	Recurse level 1	[[1]]
i = 1	Exclude 2	Base case hit	Add subset [1] to results

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **State Reversals:** Essential backtrack actions pop choice selections to restore node states before adjacent searches.
- **Pruning:** Skip duplicate recursive paths by checking elements duplicates before branching.
- **Related LeetCode Problems:** #78 Subsets, #90 Subsets II, #46 Permutations.